

Module 3

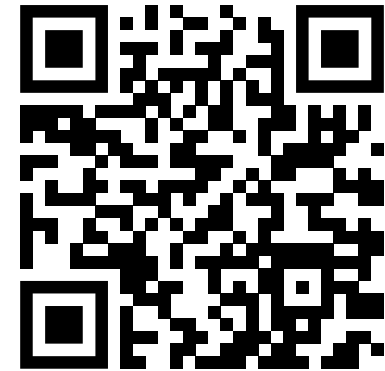
Accessibility in Jetpack Compose

Ömer Karaca

Co-Founder at Wite

Course Modules

1. *Architecture & Design Patterns*
2. *Kotlin Deep Dive*
3. **Jetpack Compose & Accessibility**
4. Networking, Data & Persistence
5. Performance, Optimization & Background Work
6. Security & Testing
7. Publishing, DevOps & Monetization



github.com/witetech/nami-android

Why Accessibility?

- **Reach** — ~16% of users have a disability; also covers *situational* impairments
- **Drivers** — legal (ADA / EN 301 549), Play Store quality, better UX for everyone
- **Agent-ready** — AI agents read the same tree via `AccessibilityService` ; deterministic and cheap, where empty semantics force a fallback to brittle vision
- **Testable by default** — the semantics tree is what Compose UI tests query; good labels double as stable test selectors
- **Wider input reach** — keyboard, D-pad/TV, Switch Access & Voice Access all ride the same tree — one investment, more devices

Android Built-in Accessibility Services

- **TalkBack** — screen reader; spoken feedback for every interaction
- **Select to Speak** — reads selected on-screen text aloud on demand
- **Switch Access** — scans focusable, actionable nodes; physical switches trigger them
- **Voice Access** — maps spoken labels to `contentDescription`; wrong labels break it
- **Keyboard / D-pad** — `focusable`, `focusGroup`, sane focus order; used on TV & external keyboards
- **Live Transcribe / Live Caption** — real-time speech-to-text overlays
- **Sound Amplifier · Color correction/inversion · Magnification · Font & display size · Extra dim**

The Semantics Tree

- Compose builds a **parallel tree** of *meaning* — what a11y services consume
- A node is a **bag of properties**: `Role`, `contentDescription`, `stateDescription`, `onClick`, `disabled` ...
- Two views: **merged** (TalkBack) vs **unmerged** (testing)
- **Classify first**: Material is free, composite needs merging, custom needs a role, decorative gets cleared

```
// Material → semantics for free
Button(onClick = { }) { Text("Save") }
// "Save, button"

// Raw layout → empty node
Box(Modifier.clickable { }) { Icon(...) }
// nothing to announce

// See what services actually see
composeTestRule.onRoot()
    .printToLog("SEMANTICS")
```

Modifier.semantics

- The tool for **adding meaning** to custom elements
- `role` — what it *is* (`Button` , `Image` ...)
- `contentDescription` — the spoken **label**
- `stateDescription` — wording for state
- `heading()` — landmark for quick jumps
- `disabled()` — announced as unavailable
- `onClick(label)` — names the action verb
- Decorative? `contentDescription = null`
- **Canvas / custom draw** — zero auto-semantics; author *everything* with `Modifier.semantics`

Demo: Content description & roles

```
Box(
    Modifier
        .clickable { onPlay() }
        .semantics {
            role = Role.Button           // what it is
            contentDescription = "Play"  // spoken label
            stateDescription = "Paused"  // current state
            heading()                    // section landmark
            disabled()                   // not actionable
            onClick(label = "Play") {    // action verb
                onPlay(); true
            }
        }
)

// Decorative image → pruned from the tree
Image(painter = bg, contentDescription = null)
```

Grouping & Hiding Nodes

- A card of 5 pieces = **5 swipes** by default
- `mergeDescendants` — read it as **one focusable unit**
- `clearAndSetSemantics` — drop noisy children, set **one clean label**
- Pitfall: **over-merging** buries nested buttons
→ keep real actions separate

Demo: Merging & hiding nodes

```
@Composable
fun PersonCard(person: Person) {
    Column(
        Modifier.clearAndSetSemantics {
            contentDescription =
                "${person.name}, age ${person.age}"
        }
    ) {
        Text(person.name)           // "Ada Lovelace"
        Text("${person.age} yrs")   // "36 yrs"
        Icon(person.flag, null)     // decorative
    }
}

// Default: "Ada Lovelace" · "36 yrs" (2 stops)
// Now:     "Ada Lovelace, age 36" (1 clean stop)
```

State & Toggles

- Use `Modifier.toggleable` / `selectable` + a `Role` — not a bare `clickable`
- The role makes the **state** part of the announcement automatically
- `stateDescription` — override the wording ("On" / "Off")
- A silent custom toggle just says *"button"* — the user can't tell on from off

Demo: State & toggles

```
// Bad: bare clickable → TalkBack says "button", no state
Modifier.clickable { enabled = !enabled }

// Good: toggleable + stateDescription
Modifier
    .toggleable(value = enabled, role = Role.Switch) { enabled = it }
    .semantics { stateDescription = if (enabled) "On" else "Off" }
```


Sliders & Progress

- Material `Slider` / progress bars announce value **and** accept adjustment for free
- A custom-drawn track is just *"button"* — no value, no way to change it
- `progressBarRangeInfo` — current value, range, discrete steps
- `setProgress` — TalkBack & Switch Access can **move** it (volume keys)
- `stateDescription` — humanize it: *"60%", "Medium"*
- A track with a child must **merge** to be focusable — plus a `contentDescription` name

Demo: Sliders & progress

```
// Custom volume track → author the value yourself.
// Box has a child fill → merge, or TalkBack never focuses it.
Box(
    Modifier.semantics(mergeDescendants = true) {
        contentDescription = "Volume" // name, or it's silent
        progressBarRangeInfo = ProgressBarRangeInfo(
            current = volume,           // 0f..1f
            range = 0f..1f,
            steps = 9,                  // 0, 10 ... 100%
        )
        stateDescription = "${(volume * 100).roundToInt()}%"
        setProgress { target ->        // volume keys / rotor
            volume = target; true
        }
    }
)
```

TalkBack: Traversal & Focus

- Default reading order follows **layout order**
- An **alert** below the article is heard *last* — too late
- `isTraversalGroup` — scope ordering to a section
- `traversalIndex` — lower sorts earlier (`-1f` = first)
- Index must sit on the **focusable** (merged) node
- Verify by **swiping**, not tapping

Demo: Traversal & focus order

```
// AlertBanner sits below the article, so TalkBack
// reaches it last – but it's urgent. Read it first.
Column(
    Modifier.semantics { isTraversalGroup = true }
) {
    Article()
    AlertBanner(
        Modifier.semantics(mergeDescendants = true) {
            traversalIndex = -1f // pull to front
        }
    )
}
```

Keyboard Focus ≠ TalkBack Order

- Two **independent** systems — fixing one doesn't fix the other
- TalkBack order → `traversalIndex`, `isTraversalGroup`
- Keyboard / D-pad → `focusable`, `focusGroup()`, `focusProperties { next = ... }`
- Verify with **Tab and swipe** — they can disagree
- Keyboard users need a **visible focus ring** (3:1 non-text contrast)

```
val (email, pwd) = remember { FocusRequester.createRefs() }

// Keyboard order – separate from TalkBack traversal
TextField(
    value = ..., onValueChange = ...,
    modifier = Modifier
        .focusRequester(email)
        .focusProperties { next = pwd } // Tab → password
)
TextField(
    value = ..., onValueChange = ...,
    modifier = Modifier.focusRequester(pwd),
)
```

Lists & Collections

- `LazyColumn` / `LazyRow` emit collection semantics **for free**
- Off-screen Lazy items **aren't in the tree** — don't assert on them
- A plain `Column` used as a list announces **no position**
- `collectionInfo` + `collectionItemInfo` → "2 of 5"
- Merge each row into **one** stop (see *Grouping*)

Demo: Lists & collections

```
// Custom (non-lazy) list → author the collection yourself
Column(
    Modifier.semantics {
        collectionInfo = CollectionInfo(rowCount = items.size, columnCount = 1)
    }
) {
    items.forEachIndexed { i, item ->
        Row(
            Modifier.semantics(mergeDescendants = true) {
                collectionItemInfo =
                    CollectionItemInfo(i, 1, 0, 1) // row, span, col, span
            }
        ) { /* ... */ }
    }
}
// TalkBack: "Inbox, 12, 2 of 5"
```

Live Announcements

- A change off-focus is **silent** for TalkBack — sighted users see it, others don't
- `liveRegion` announces a node's text changes **automatically**
- `Polite` — waits for a pause; `Assertive` — interrupts now
- Errors → `Assertive`; loading/status → `Polite`
- Don't spam: it re-announces on **every change**

Demo: Announcements (liveRegion)

```
var status by remember { mutableStateOf("Idle") }
var loadId by remember { mutableIntStateOf(0) }

LaunchedEffect(loadId) {
    if (loadId == 0) return@LaunchedEffect
    status = "Loading..."
    delay(1_200)
    status = "Updated just now"
}

Column {
    Button(onClick = { loadId++ }) { Text("Reload") }
    Text(
        text = status,
        // Announce changes without stealing focus
        modifier = Modifier.semantics {
            liveRegion = LiveRegionMode.Polite
        },
    )
}
```

Forms & Validation

- Label + field + error should read as **one** node, not three
- `error("...")` — TalkBack announces *"invalid entry"* + message
- A red border is **silent** — announce on submit with `liveRegion`
- `password()` — masks the spoken characters
- Material `isError` + `supportingText` wire much of this for you

Demo: Forms & validation

```
OutlinedTextField(  
    value = email, onValueChange = { email = it },  
    label = { Text("Email") },  
    isError = emailError != null,  
    supportingText = {  
        emailError?.let {  
            Text(it, Modifier.semantics {  
                liveRegion = LiveRegionMode.Assertive // speak it now  
            })  
        }  
    },  
    // Field reads as "Email, invalid entry"  
    modifier = Modifier.semantics {  
        emailError?.let { error(it) }  
    },  
)
```

Panes & Screen Changes

- Navigating screens or opening a dialog is **silent** to TalkBack
- `paneTitle` — names a region that appears in place (sheet, side pane)
- `dismiss { ... }` — close a sheet / snackbar from the rotor
- Move focus to the new heading:
`FocusRequester.requestFocus()`
- Dialogs: a title node **and** a sensible first focus

```
// A bottom sheet / overlay that appears in place
ModalBottomSheet(onDismissRequest = ::close) {
    Column(
        Modifier.semantics {
            paneTitle = "Filters"           // announced on appear
            dismiss { close(); true }      // rotor: "Dismiss"
        }
    ) {
        FilterControls()
    }
}
```

Color Contrast

- **WCAG AA:** 4.5:1 body text, 3:1 large text & UI components
- **Never color alone** — color-blind users & screen readers miss it
- Pair state with an **icon, shape, or label**
- Verify **both** light *and* dark schemes
- M3 **tonal elevation** lightens dark surfaces
→ contrast can silently drop

```
// Bad: error shown only as red text
Text(label, color = colorScheme.error)

// Good: color + icon + text – redundant cues
Row(verticalAlignment = CenterVertically) {
    Icon(Icons.Default.Error, contentDescription = null)
    Text("$label – required", color = colorScheme.error)
}
```


Font Scaling

- `sp` for **text**, `dp` for **layout** — never text in `dp`
- Android 14+ scales **non-linearly up to 200%** — test at max
- **Fixed heights clip** scaled text — let containers wrap
- Users set this in Settings ▶ Display ▶ Font size
- **RTL & localization** — use `start / end` not `left / right` ; translated strings expand — test both

Demo: Font scaling

```
// Bad: fixed 40dp height clips scaled text
Surface(Modifier.fillMaxWidth().height(40.dp)) {
    Box(contentAlignment = Center) {
        Text("Confirm purchase")
    }
}

// Good: padding instead of height → it grows
Surface(Modifier.fillMaxWidth()) {
    Text(
        "Confirm purchase",
        Modifier.padding(vertical = 12.dp, horizontal = 16.dp),
    )
}
```

Touch Targets

- **48×48dp minimum** for anything tappable
- The *icon* can stay small — the **hit area** can't
- `minimumInteractiveComponentSize()` expands it without resizing
- **Spacing** between targets matters as much as size

Demo: Touch targets

```
// Bad: hit area == the 24dp glyph
Icon(
    Icons.Filled.Close, "Dismiss",
    Modifier.size(24.dp).clickable {}
)

// Good: icon stays 24dp, touch area ≥ 48dp
Icon(
    Icons.Filled.Close, "Dismiss",
    Modifier
        .minimumInteractiveComponentSize()
        .size(24.dp)
        .clickable {}
)
```

Adapting to A11y Settings

- Read the user's settings — don't assume the defaults
- `calculateRecommendedTimeoutMillis()` — **extend** Snackbars / auto-dismiss when TalkBack is on
- Detect a screen reader → drop hover-only or auto-timed UX
- Honor **reduced motion** (animator scale 0) system-wide
- One adaptive layer benefits every assistive tool

```
val a11y = LocalAccessibilityManager.current

// A fixed 4s Snackbar is unreadable with TalkBack – let the
// system stretch it (returns a longer value, or original).
val timeout = a11y?.calculateRecommendedTimeoutMillis(
    originalTimeoutMillis = 4_000,
    containsIcons = false,
    containsText = true,
    containsControls = true,
) ?: 4_000L

LaunchedEffect(message) {
    snackbarHostState.showSnackbar(message) // honor `timeout`
}
```

Gestures & Custom Actions

- Custom gestures (drag/swipe) are **invisible** to a11y
- TalkBack **eats swipes** for its own navigation
- `customActions` surfaces them in the **rotor menu**
- Always offer a **non-gesture path**
- Respect *"Remove animations"* (scale = 0)

Demo: Custom actions for gestures

```
// Swipe-to-delete: gesture in pointerInput,  
// invisible to TalkBack. Expose it as an action.  
Modifier.semantics {  
    customActions = listOf(  
        CustomAccessibilityAction("Delete") {  
            onDelete()  
            true // handled  
        }  
    )  
}
```

Inspect & Debug

- **Layout Inspector** now shows semantics per node
- `printToLog()` — dump the **merged** vs **unmerged** tree
- "See what TalkBack sees" *before* fixing
- Debug the **tree**, not the UI
- **AndroidView interop** — semantics must bridge the legacy-View gap; test the boundary explicitly

```
@Test
fun dumpTree() {
    composeTestRule.setContent { PersonCard(ada) }

    // Merged tree – what TalkBack consumes
    composeTestRule.onRoot(useUnmergedTree = false)
        .printToLog("MERGED")

    // Unmerged tree – every node, for assertions
    composeTestRule.onRoot(useUnmergedTree = true)
        .printToLog("UNMERGED")
}
```

Testing: Automated

- `enableAccessibilityChecks()` flags issues in UI tests
- Semantics matchers:
 - `onNodeWithContentDescription`,
 - `hasRole`
- Espresso
 - `AccessibilityChecks.enable()` for legacy/Views
- Wire it into **CI** — catch regressions on every PR

```
@Test
fun sendButton_isAccessible() {
    composeTestRule.enableAccessibilityChecks()
    composeTestRule.setContent { ChatBar() }

    composeTestRule
        .onNodeWithContentDescription("Send message")
        .assertHasClickAction()
    // a11y checks now run on every interaction
}
```

Testing: Tooling & Manual

- **Accessibility Scanner** app — contrast, target size, missing labels on a real screen
- **Manual TalkBack pass** — swipe through; the only real test of the *experience*
- CI gap: automation catches **structure**, never **usability**

Thank you

Build the semantics tree on purpose, not by accident.